In [1]:
```
import pandas as pd
import numpy as np

data = {
    "order_id": [
        101, 102, 103, 104, 105,
        106, 107, 108, 109, 110,
        111, 112, 113, 114, 115,
        116, 117, 101, 119, 120
    ],
    "product": [
        "Laptop", "Mouse", "Keyboard", "Monitor", "Phone",
        "Tablet", "Headphones", "Printer", "Laptop", "Mouse",
        "Smartwatch", "Camera", "Keyboard", "Speaker", "Phone",
        "Monitor", "Tablet", "Laptop", "Camera", "Speaker"
    ],
    "city": [
        "Chennai", "Hyderabad", "Bangalore", "Chennai", None,
        "Mumbai", "Hyderabad", "Delhi", "Mumbai", "Chennai",
        "Delhi", "Bangalore", "Chennai", "Mumbai", "Hyderabad",
        None, "Delhi", "Chennai", "Bangalore", "Mumbai"
    ],
    "quantity": [
        1, 2, 1, 2, 1,
        3, 2, 1, 1, 4,
        1, 2, None, 2, 1,
        2, 1, 1, 2, 3
    ],
    "unit_price": [
        55000, 500, 1500, 12000, 65000,
        25000, 2500, 8000, 55000, 500,
        12000, 30000, 1500, 3500, 65000,
        12000, 25000, 55000, "invalid", -3500
    ],
    "order_date": [
        "2026-01-01", "2026-01-02", "2026-01-03",
        "2026-01-04", "2026-01-05", "2026-01-06",
        "2026-01-07", None, "2026-01-09",
        "2026-01-10", "2026-01-11", "2026-01-12",
        "2026-01-13", "2026-01-14", "2026-01-15",
        "2026-01-16", "2026-01-17", "2026-01-01",
        "2026-01-19", "2026-01-20"
    ]
}

df = pd.DataFrame(data)

print(df)
```

	order_id	product	city	quantity	unit_price	order_date
0	101	Laptop	Chennai	1.0	55000	2026-01-01
1	102	Mouse	Hyderabad	2.0	500	2026-01-02
2	103	Keyboard	Bangalore	1.0	1500	2026-01-03
3	104	Monitor	Chennai	2.0	12000	2026-01-04
4	105	Phone	None	1.0	65000	2026-01-05
5	106	Tablet	Mumbai	3.0	25000	2026-01-06
6	107	Headphones	Hyderabad	2.0	2500	2026-01-07
7	108	Printer	Delhi	1.0	8000	None
8	109	Laptop	Mumbai	1.0	55000	2026-01-09
9	110	Mouse	Chennai	4.0	500	2026-01-10
10	111	Smartwatch	Delhi	1.0	12000	2026-01-11
11	112	Camera	Bangalore	2.0	30000	2026-01-12
12	113	Keyboard	Chennai	NaN	1500	2026-01-13
13	114	Speaker	Mumbai	2.0	3500	2026-01-14
14	115	Phone	Hyderabad	1.0	65000	2026-01-15
15	116	Monitor	None	2.0	12000	2026-01-16
16	117	Tablet	Delhi	1.0	25000	2026-01-17
17	101	Laptop	Chennai	1.0	55000	2026-01-01
18	119	Camera	Bangalore	2.0	invalid	2026-01-19
19	120	Speaker	Mumbai	3.0	-3500	2026-01-20

In [2]:
```
df.isnull()
```

Out[2]:

	order_id	product	city	quantity	unit_price	order_date
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	True	False	False	False
5	False	False	False	False	False	False
6	False	False	False	False	False	False
7	False	False	False	False	False	True
8	False	False	False	False	False	False
9	False	False	False	False	False	False
10	False	False	False	False	False	False
11	False	False	False	False	False	False
12	False	False	False	True	False	False
13	False	False	False	False	False	False
14	False	False	False	False	False	False
15	False	False	True	False	False	False
16	False	False	False	False	False	False
17	False	False	False	False	False	False
18	False	False	False	False	False	False
19	False	False	False	False	False	False

In [3]:
```
df.isnull().sum()
```

Out[3]:
```
order_id      0
product       0
city          2
quantity      1
unit_price    0
order_date    1
dtype: int64
```

In [4]:
```
df.isnull().sum().sum()
```

Out[4]:
```
4
```

In [5]:
```
df[df.duplicated(subset="order_id", keep=False)]
```

	order_id	product	city	quantity	unit_price	order_date
0	101	Laptop	Chennai	1.0	55000	2026-01-01
17	101	Laptop	Chennai	1.0	55000	2026-01-01

In [6]:
```
df.duplicated("order_id").sum()
```

Out[6]:
```
1
```

In [7]:
```
df["unit_price"] = pd.to_numeric(df["unit_price"],errors="coerce")
valid_prices = df[df["unit_price"].notna()]
highest_price = valid_prices["unit_price"].max()

highest_product = valid_prices[valid_prices["unit_price"] == highest_price]

print("Highest unit price:", highest_price)
print(highest_product[["product", "unit_price", "order_id"]])
```

```
Highest unit price: 65000.0
   product  unit_price  order_id
4    Phone     65000.0       105
14   Phone     65000.0       115
```

In [ ]:
```

```

In [8]:
```
df["unit_price"] = pd.to_numeric(df["unit_price"], errors="coerce")
df["quantity"] = pd.to_numeric(df["quantity"], errors="coerce")

df.loc[df["unit_price"] < 0, "unit_price"] = np.nan
df["revenue"] = df["quantity"] * df["unit_price"]
revenue_by_city = df.groupby("city", dropna=False)["revenue"].sum()
revenue_by_city
```

Out[8]:
```
city
Bangalore     61500.0
Chennai      136000.0
Delhi         45000.0
Hyderabad     71000.0
Mumbai       137000.0
NaN           89000.0
Name: revenue, dtype: float64
```

Basic aggregation

In [9]:
```
print("Total Revenue:", df["revenue"].sum())
print("Average Unit Price:", df["unit_price"].mean())
print("Maximum Unit Price:", df["unit_price"].max())
print("Minimum Unit Price:", df["unit_price"].min())
print("Total Quantity Sold:", df["quantity"].sum())
print("Number of Orders:", df["order_id"].nunique())
```

```
Total Revenue: 539500.0
Average Unit Price: 23833.333333333332
Maximum Unit Price: 65000.0
Minimum Unit Price: 500.0
Total Quantity Sold: 33.0
Number of Orders: 19
```

In [11]:
```
df.agg({
    "quantity": ["sum", "mean", "min", "max"],
    "unit_price": ["mean", "min", "max"],
    "revenue": ["sum", "mean", "max"]
})
```

Out[11]:

	quantity	unit_price	revenue
sum	33.000000	NaN	539500.000000
mean	1.736842	23833.333333	31735.294118
min	1.000000	500.000000	NaN
max	4.000000	65000.000000	75000.000000

Revenue by city

In [13]:
```
revenue_by_city = df.groupby("city")["revenue"].sum()
revenue_by_city
```

```
city
Bangalore     61500.0
Chennai      136000.0
Delhi         45000.0
Hyderabad     71000.0
Mumbai       137000.0
Name: revenue, dtype: float64
```

Total quantity sold by product

In [14]:
```
quantity_by_product = df.groupby("product")["quantity"].sum()

print(quantity_by_product.sort_values(ascending=False))
```

```
product
Mouse         6.0
Speaker       5.0
Camera        4.0
Monitor       4.0
Tablet        4.0
Laptop        3.0
Headphones    2.0
Phone         2.0
Keyboard      1.0
Printer       1.0
Smartwatch    1.0
Name: quantity, dtype: float64
```

Multiple aggregations by city

In [15]:
```
city_summary = df.groupby("city").agg(
    total_revenue=("revenue", "sum"),
    average_price=("unit_price", "mean"),
    total_quantity=("quantity", "sum"),
    order_count=("order_id", "nunique")
)

print(city_summary)
```

```
           total_revenue  average_price  total_quantity  order_count
city
Bangalore        61500.0   15750.000000             5.0            3
Chennai         136000.0   24800.000000             8.0            4
Delhi            45000.0   15000.000000             3.0            3
Hyderabad        71000.0   22666.666667             5.0            3
Mumbai          137000.0   27833.333333             9.0            4
```

Group by both city and product

In [16]:
```
city_product = df.groupby(["city", "product"])["revenue"].sum()
city_product
```

Out[16]:
```
city       product
Bangalore  Camera         60000.0
           Keyboard        1500.0
Chennai    Keyboard           0.0
           Laptop        110000.0
           Monitor        24000.0
           Mouse           2000.0
Delhi      Printer         8000.0
           Smartwatch     12000.0
           Tablet         25000.0
Hyderabad  Headphones       5000.0
           Mouse           1000.0
           Phone          65000.0
Mumbai     Laptop         55000.0
           Speaker         7000.0
           Tablet         75000.0
Name: revenue, dtype: float64
```

Extract year, month, and day

In [17]:
```
df["year"] = df["order_date"].dt.year
df["month"] = df["order_date"].dt.month
df["day"] = df["order_date"].dt.day
df["day_name"] = df["order_date"].dt.day_name()
df[["order_date", "year", "month", "day", "day_name"]]
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[17], line 1
----> 1 df["year"] = df["order_date"].dt.year
      2 df["month"] = df["order_date"].dt.month
      3 df["day"] = df["order_date"].dt.day

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:6318, in NDFrame._
_getattr__(self, name)
   6311 if (
   6312     name not in self._internal_names_set
   6313     and name not in self._metadata
   6314     and name not in self._accessors
   6315     and self._info_axis._can_hold_identifiers_and_holds_name(name)
   6316 ):
   6317     return self[name]
-> 6318 return object.__getattribute__(self, name)

File ~\anaconda3\Lib\site-packages\pandas\core\accessor.py:224, in CachedAcc
essor.__get__(self, obj, cls)
    221 if obj is None:
    222     # we're accessing the attribute of the class, i.e., Dataset.geo
    223     return self._accessor
--> 224 accessor_obj = self._accessor(obj)
    225 # Replace the property with the accessor object. Inspired by:
    226 # https://www.pydanny.com/cached-property.html (https://www.pydanny.
com/cached-property.html)
    227 # We need to use object.__setattr__ because we overwrite __setattr__
on
    228 # NDFrame
    229 object.__setattr__(obj, self._name, accessor_obj)

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\accessors.py:643, in
CombinedDatetimelikeProperties.__new__(cls, data)
    640 elif isinstance(data.dtype, PeriodDtype):
    641     return PeriodProperties(data, orig)
--> 643 raise AttributeError("Can only use .dt accessor with datetimelike va
lues")

AttributeError: Can only use .dt accessor with datetimelike values
```

In [18]:
```
df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce")
```

In [20]:
```
df["order_date"].dtype
```

Out[20]:
```
dtype('<M8[ns]')
```

In [21]:
```
df["year"] = df["order_date"].dt.year
df["month"] = df["order_date"].dt.month
df["day"] = df["order_date"].dt.day
df["day_name"] = df["order_date"].dt.day_name()

print(df[["order_date", "year", "month", "day", "day_name"]])
```

```
   order_date    year  month   day   day_name
0  2026-01-01  2026.0    1.0   1.0   Thursday
1  2026-01-02  2026.0    1.0   2.0     Friday
2  2026-01-03  2026.0    1.0   3.0   Saturday
3  2026-01-04  2026.0    1.0   4.0     Sunday
4  2026-01-05  2026.0    1.0   5.0     Monday
5  2026-01-06  2026.0    1.0   6.0    Tuesday
6  2026-01-07  2026.0    1.0   7.0  Wednesday
7         NaT     NaN    NaN   NaN        NaN
8  2026-01-09  2026.0    1.0   9.0     Friday
9  2026-01-10  2026.0    1.0  10.0   Saturday
10 2026-01-11  2026.0    1.0  11.0     Sunday
11 2026-01-12  2026.0    1.0  12.0     Monday
12 2026-01-13  2026.0    1.0  13.0    Tuesday
13 2026-01-14  2026.0    1.0  14.0  Wednesday
14 2026-01-15  2026.0    1.0  15.0   Thursday
15 2026-01-16  2026.0    1.0  16.0     Friday
16 2026-01-17  2026.0    1.0  17.0   Saturday
17 2026-01-01  2026.0    1.0   1.0   Thursday
18 2026-01-19  2026.0    1.0  19.0     Monday
19 2026-01-20  2026.0    1.0  20.0    Tuesday
```

Filter orders between two dates

In [22]:
```
start_date = "2026-01-05"
end_date = "2026-01-15"
filtered = df[(df["order_date"]>=start_date)&(df["order_date"]<=end_date)]
filtered
```

Out[22]:

	order_id	product	city	quantity	unit_price	order_date	revenue	year	month
4	105	Phone	None	1.0	65000.0	2026-01-05	65000.0	2026.0	1.0
5	106	Tablet	Mumbai	3.0	25000.0	2026-01-06	75000.0	2026.0	1.0
6	107	Headphones	Hyderabad	2.0	2500.0	2026-01-07	5000.0	2026.0	1.0
8	109	Laptop	Mumbai	1.0	55000.0	2026-01-09	55000.0	2026.0	1.0
9	110	Mouse	Chennai	4.0	500.0	2026-01-10	2000.0	2026.0	1.0
10	111	Smartwatch	Delhi	1.0	12000.0	2026-01-11	12000.0	2026.0	1.0
11	112	Camera	Bangalore	2.0	30000.0	2026-01-12	60000.0	2026.0	1.0
12	113	Keyboard	Chennai	NaN	1500.0	2026-01-13	NaN	2026.0	1.0
13	114	Speaker	Mumbai	2.0	3500.0	2026-01-14	7000.0	2026.0	1.0
14	115	Phone	Hyderabad	1.0	65000.0	2026-01-15	65000.0	2026.0	1.0

Daily revenue

In [23]:
```
daily_revenue = df.groupby("order_date")["revenue"].sum()
daily_revenue
```

Out[23]:
```
order_date
2026-01-01    110000.0
2026-01-02      1000.0
2026-01-03      1500.0
2026-01-04     24000.0
2026-01-05     65000.0
2026-01-06     75000.0
2026-01-07      5000.0
2026-01-09     55000.0
2026-01-10      2000.0
2026-01-11     12000.0
2026-01-12     60000.0
2026-01-13         0.0
2026-01-14      7000.0
2026-01-15     65000.0
2026-01-16     24000.0
2026-01-17     25000.0
2026-01-19         0.0
2026-01-20         0.0
Name: revenue, dtype: float64
```

Monthly revenue

In [24]:
```
monthly_revenue = df.groupby( df["order_date"].dt.to_period("M"))["revenue"].s
monthly_revenue
```

Out[24]:
```
order_date
2026-01    531500.0
Freq: M, Name: revenue, dtype: float64
```

Find orders placed on weekends

In [25]:
```
weekend_orders = df[df["order_date"].dt.dayofweek >= 5]
weekend_orders
```

Out[25]:

	order_id	product	city	quantity	unit_price	order_date	revenue	year	month	c
2	103	Keyboard	Bangalore	1.0	1500.0	2026-01-03	1500.0	2026.0	1.0	
3	104	Monitor	Chennai	2.0	12000.0	2026-01-04	24000.0	2026.0	1.0	
9	110	Mouse	Chennai	4.0	500.0	2026-01-10	2000.0	2026.0	1.0	1
10	111	Smartwatch	Delhi	1.0	12000.0	2026-01-11	12000.0	2026.0	1.0	1
16	117	Tablet	Delhi	1.0	25000.0	2026-01-17	25000.0	2026.0	1.0	1

filter selected cities

In [26]:
```
selected_cities = df[df["city"].isin(["Chennai", "Mumbai", "Hyderabad"])]
selected_cities
```

Out[26]:

	order_id	product	city	quantity	unit_price	order_date	revenue	year	month
0	101	Laptop	Chennai	1.0	55000.0	2026-01-01	55000.0	2026.0	1.0
1	102	Mouse	Hyderabad	2.0	500.0	2026-01-02	1000.0	2026.0	1.0
3	104	Monitor	Chennai	2.0	12000.0	2026-01-04	24000.0	2026.0	1.0
5	106	Tablet	Mumbai	3.0	25000.0	2026-01-06	75000.0	2026.0	1.0
6	107	Headphones	Hyderabad	2.0	2500.0	2026-01-07	5000.0	2026.0	1.0
8	109	Laptop	Mumbai	1.0	55000.0	2026-01-09	55000.0	2026.0	1.0
9	110	Mouse	Chennai	4.0	500.0	2026-01-10	2000.0	2026.0	1.0
12	113	Keyboard	Chennai	NaN	1500.0	2026-01-13	NaN	2026.0	1.0
13	114	Speaker	Mumbai	2.0	3500.0	2026-01-14	7000.0	2026.0	1.0
14	115	Phone	Hyderabad	1.0	65000.0	2026-01-15	65000.0	2026.0	1.0
17	101	Laptop	Chennai	1.0	55000.0	2026-01-01	55000.0	2026.0	1.0
19	120	Speaker	Mumbai	3.0	NaN	2026-01-20	NaN	2026.0	1.0

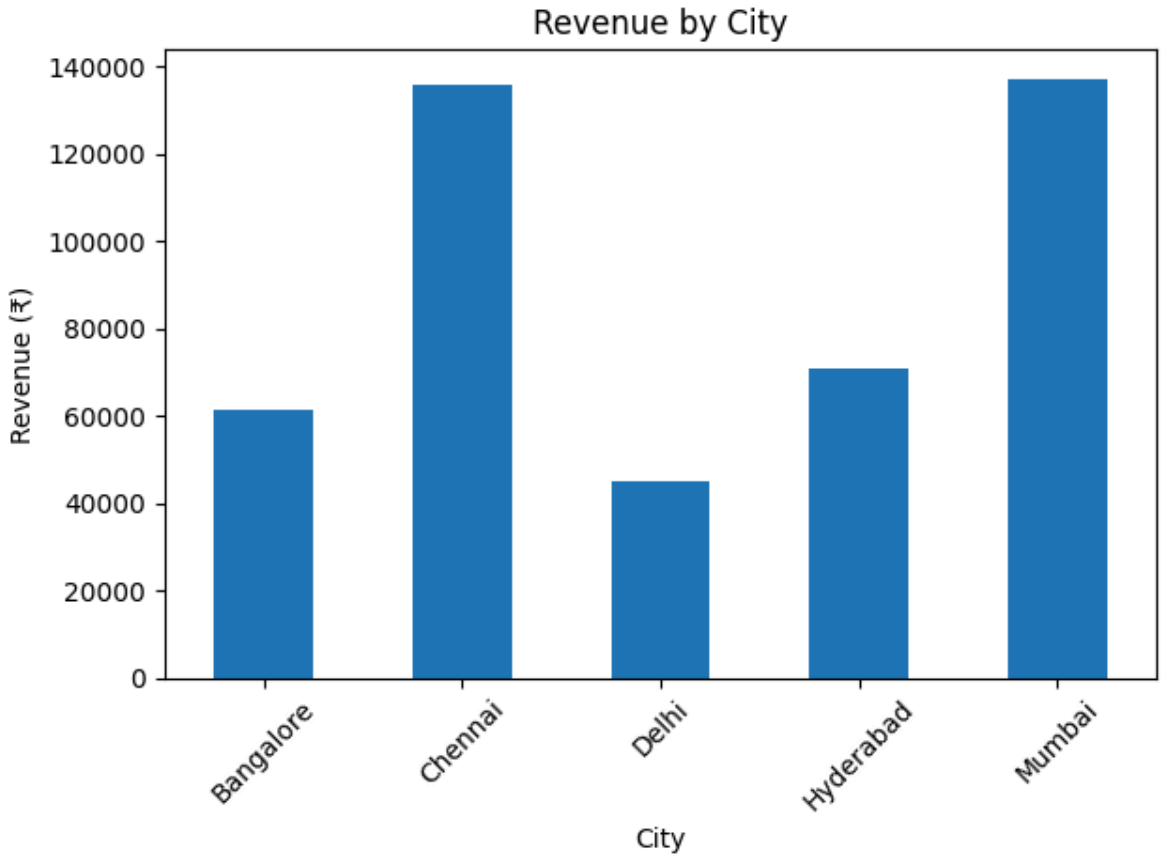
charts using Matplotlib

In [27]:
```
import matplotlib.pyplot as plt
```

revenue by city

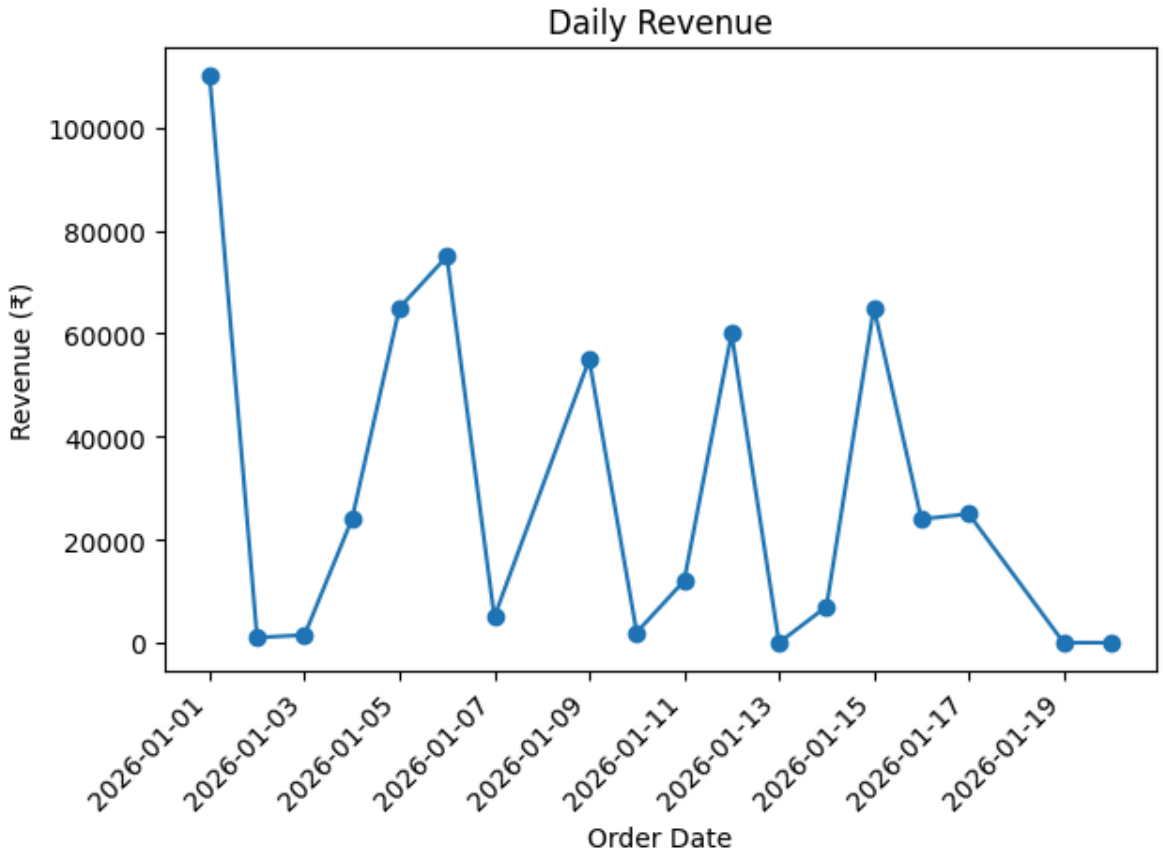
In [28]:
```
revenue_by_city = df.groupby("city")["revenue"].sum()
revenue_by_city.plot(kind="bar")

plt.title("Revenue by City")
plt.xlabel("City")
plt.ylabel("Revenue (₹)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



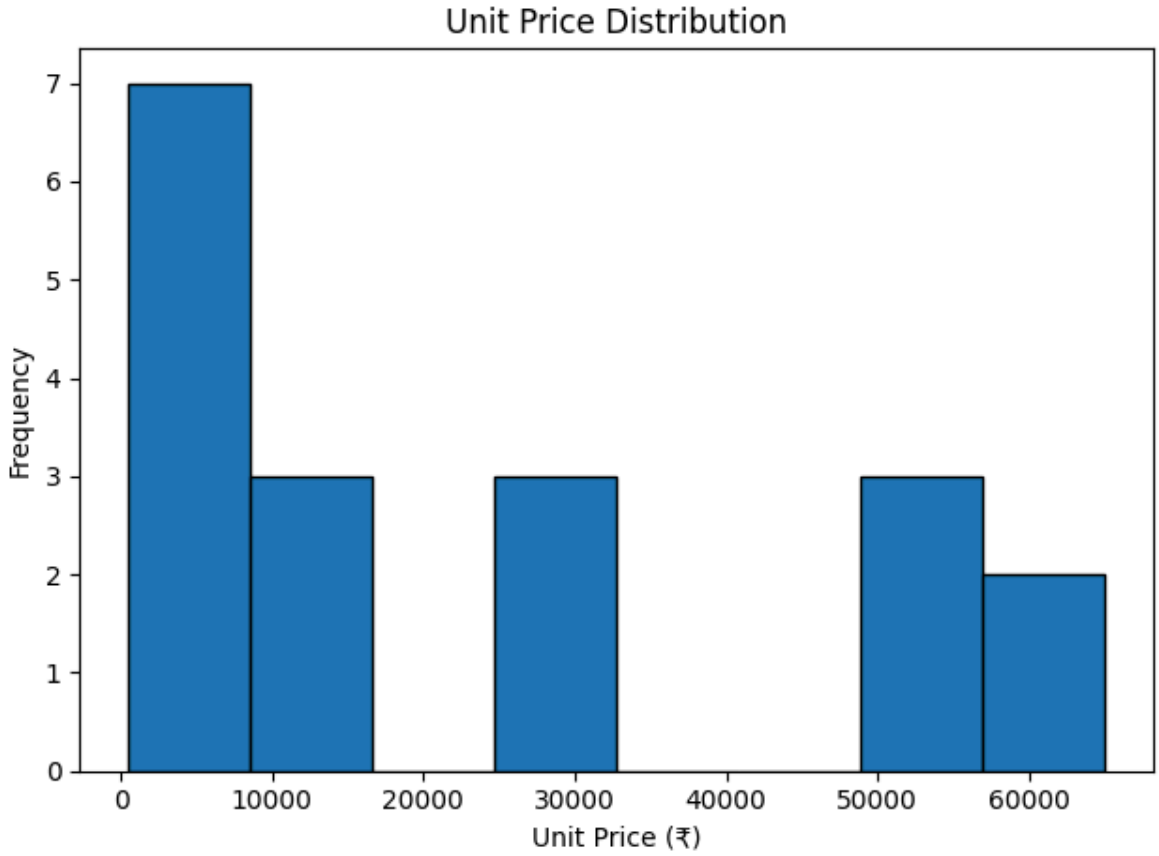
daily revenue

In [29]:
```
daily_revenue = df.groupby("order_date")["revenue"].sum()
daily_revenue.plot(kind="line", marker="o")
plt.title("Daily Revenue")
plt.xlabel("Order Date")
plt.ylabel("Revenue (₹)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



distribution of unit prices

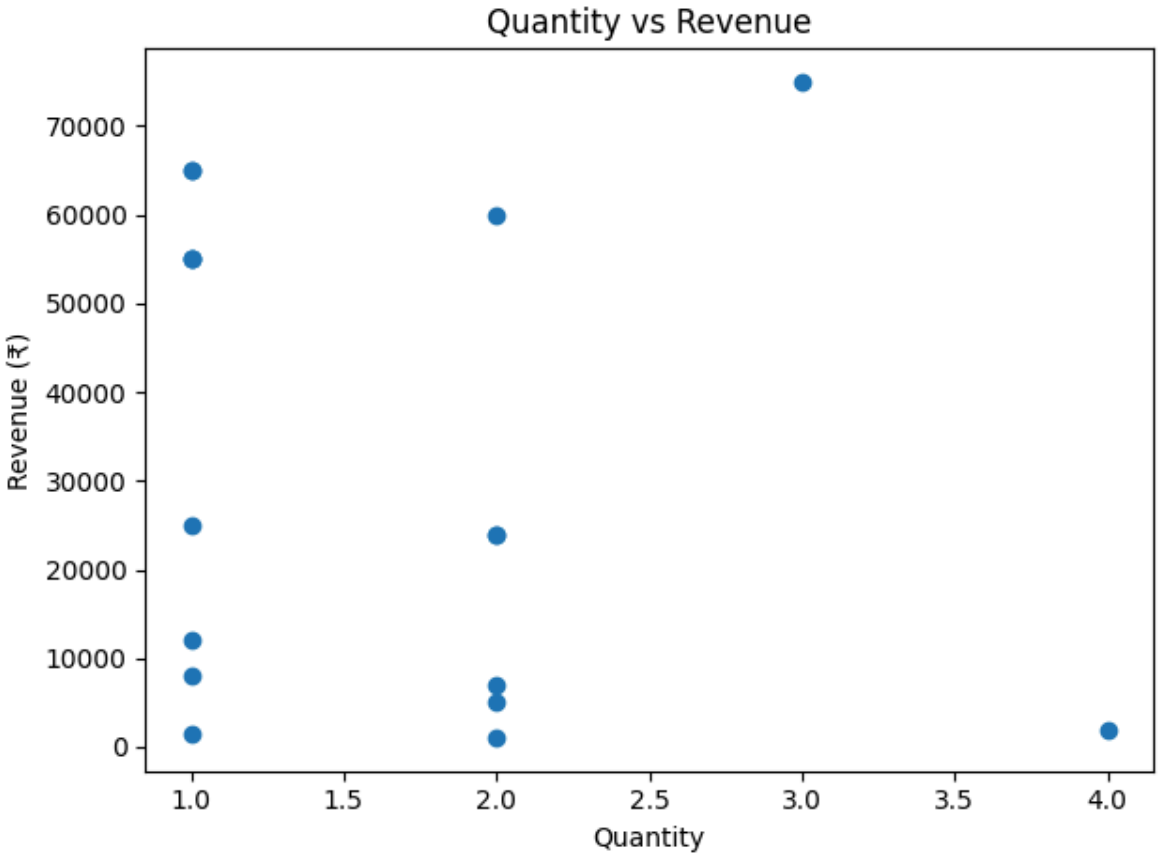
In [30]:
```
df["unit_price"].dropna().plot(kind="hist",bins=8,edgecolor="black")
plt.title("Unit Price Distribution")
plt.xlabel("Unit Price (₹)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```



quantity versus revenue

In [31]:
```
plt.scatter(df["quantity"], df["revenue"])

plt.title("Quantity vs Revenue")
plt.xlabel("Quantity")
plt.ylabel("Revenue (₹)")
plt.tight_layout()
plt.show()
```



In [ ]:
```

```